คู่มือ · Project II

โครงสร้างโปรเจกต์ ระบบประเมินความเสี่ยงสุขภาพ

อ่านจบแล้วรู้ว่าไฟล์ไหนอยู่ตรงไหน ทำงานต่อกันอย่างไร และถ้าต้องแก้อะไรต้องไปเปิดไฟล์ไหน

1 ภาพรวม: เทคโนโลยีและ 3 ชั้นของระบบ

2 แผนที่โฟลเดอร์

3 กฎตั้งชื่อไฟล์ของ Next.js

4 ผู้ใช้ 3 บทบาท

5 ฝั่งผู้ใช้งาน: หน้าต่าง ๆ

6 หลังบ้าน (API)

7 ไฟล์ตัวช่วยใน lib/

8 ฐานข้อมูล

9 ตามรอย 1 รอบการทำงาน

10 ความปลอดภัย

11 อยากแก้ X ไปที่ไฟล์ไหน

12 สรุปจำง่ายก่อนสอบ

Next.js 16 · React 19 · TypeScript · Tailwind CSS 4 · PostgreSQL (Supabase) · ประมาณ 176 ไฟล์

1 ภาพรวมโปรเจกต์

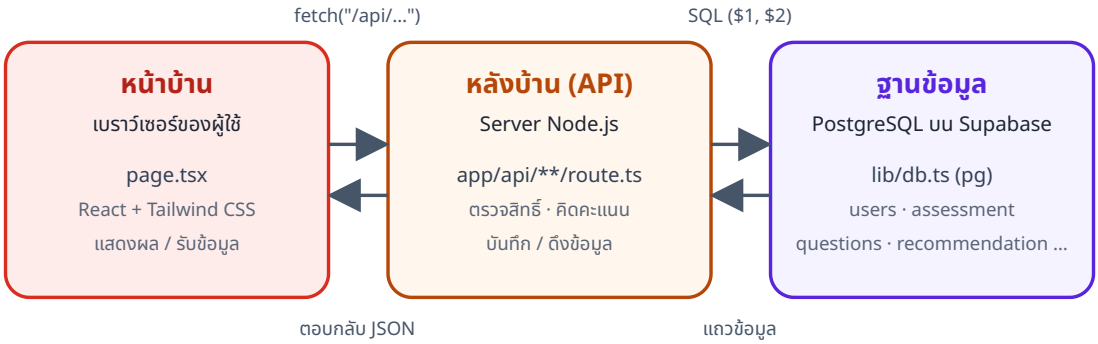
ใช้อะไรเขียน และระบบแบ่งเป็นกี่ชั้น

โปรเจกต์นี้เป็น **เว็บแอป** ให้ผู้ใช้ทำแบบประเมินความเสี่ยงสุขภาพ 12 แบบ แล้วได้ผลพร้อมคำแนะนำ มีฝั่ง **เจ้าหน้าที่** ไว้จัดการแบบประเมินและดูสถิติ และฝั่ง **ผู้ดูแลระบบ** ไว้ดูแลเซิร์ฟเวอร์กับฐานข้อมูล

เทคโนโลยีที่ใช้

ส่วน	ใช้อะไร	หน้าที่
Framework	Next.js 16 (App Router)	รวมหน้าบ้านและหลังบ้านไว้ในโปรเจกต์เดียว
หน้าบ้าน	React 19 + TypeScript	สร้างหน้าเว็บเป็นคอมโพเนนต์
ตกแต่ง	Tailwind CSS 4 + lucide-react	สไตล์ด้วย class และไอคอน
หลังบ้าน	Next.js Route Handlers	API ในไฟล์ `route.ts` ตอบเป็น JSON
ฐานข้อมูล	PostgreSQL บน Supabase	เก็บผู้ใช้ ผลประเมิน คำถาม คำแนะนำ
ต่อฐานข้อมูล	`pg` (node-postgres)	เปิดการเชื่อมต่อใน `lib/db.ts`
รหัสผ่าน	`bcryptjs`	เก็บรหัสเป็น hash ไม่เก็บรหัสจริง
Session	cookie ลงลายเซ็น HMAC	จำว่าใครล็อกอินอยู่ (`lib/session.ts`)
อีเมล OTP	`nodemailer` + Gmail	ส่งรหัส OTP ตอนลืมรหัสผ่าน

ระบบมี 3 ชั้น



หน้าบ้าน + หลังบ้าน อยู่ในโปรเจกต์ Next.js เดียวกัน (โฟลเดอร์ app/)

จำประโยคนี้ไว้ตอบอาจารย์: หน้าเว็บ (`page.tsx`) เรียก `fetch("/api/…")` ไปที่หลังบ้าน (`route.ts`) หลังบ้านตรวจสิทธิ์และคิดคะแนน แล้วคุยกับ PostgreSQL ผ่าน `lib/db.ts` จากนั้นส่ง JSON กลับมาให้หน้าเว็บแสดงผล

-

2 แผนที่โฟลเดอร์

ทุกอย่างอยู่ใน health-assessment/

เปิดโปรเจกต์ใน VS Code แล้วจะเห็นโครงนี้ โฟลเดอร์ที่ต้องรู้จักมีแค่ 4 ตัว คือ **app/**, **app/api/**, **lib/** และ **sql/**

```
health-assessment/
├── app/                ← หัวใจของโปรเจกต์: ชื่อโฟลเดอร์ = URL ของหน้าเว็บ
│   ├── page.tsx · layout.tsx · globals.css   ← หน้าแรก · กรอบทุกหน้า · CSS กลาง
│   ├── welcome/ register/ login/ …           ← หน้าบัญชีผู้ใช้
│   ├── assessment-type/ assessment-menu*/    ← เมนูเลือกแบบประเมิน
│   ├── assessment_XXX/  (12 แบบ)             ← หน้าทำแบบประเมิน
│   ├── recommendation_XXX/ (12 หน้า)         ← หน้าผลและคำแนะนำ
│   ├── dashboard/ history/ notifications/    ← ติดตามผลของผู้ใช้
│   ├── profile/ settings/                    ← ข้อมูลสุขภาพ · ตั้งค่า
│   ├── components/       ← ชิ้น UI ใช้ร่วม (Sidebar, กระดิ่ง, โครงหน้าผล)
│   ├── staff/            ← หน้าเจ้าหน้าที่ (มี components ของตัวเอง)
│   ├── admin/            ← หน้าผู้ดูแลระบบ (มี components ของตัวเอง)
│   └── api/              ← หลังบ้านทั้งหมด (route.ts)
│       ├── auth/  assessments/  dashboard/  history/
│       ├── notifications/  profile/  settings/
│       ├── staff/   ← API เจ้าหน้าที่
│       └── admin/   ← API ผู้ดูแลระบบ
├── lib/                ← ตัวช่วยฝั่ง server (db, session, mailer, กฎต่าง ๆ)
├── sql/                ← สคริปต์ SQL ที่รันเองใน Supabase
├── public/             ← รูปภาพ (logo.png)
├── .env.local          ← ค่าลับ — ห้ามแชร์ ห้ามขึ้น git
└── package.json · next.config.ts · tsconfig.json   ← ไฟล์ตั้งค่า
```

app/

หน้าเว็บ + API

ชื่อโฟลเดอร์คือ URL เช่น `app/login/` = หน้า `/login` ข้างในมี `page.tsx` เป็นตัวหน้าเว็บ

app/api/

หลังบ้าน

ทุกไฟล์ชื่อ `route.ts` ใช้รับคำขอจากหน้าเว็บ เช่น `app/api/history/route.ts` = `/api/history`

lib/

ตัวช่วยใช้ร่วม

โค้ดที่หลาย API เรียกซ้ำ เช่น ต่อฐานข้อมูล ตรวจ session ส่งอีเมล และกฎแจ้งเตือน

sql/

สคริปต์ฐานข้อมูล

คำสั่ง SQL ที่คัดลอกไปรันเองใน Supabase เช่น สร้างตารางใหม่ หรือแก้ข้อมูลเก่า

ไม่ต้องแตะ: `node_modules/` (ไลบรารีที่ npm ติดตั้ง) และ `.next/` (ไฟล์ที่ Next.js build ให้เอง) ส่วน `.env.local` เก็บรหัสฐานข้อมูลกับ secret ห้ามแชร์และห้ามส่งขึ้น git

3 กฎตั้งชื่อไฟล์ของ Next.js

รู้ 5 ข้อนี้ จะอ่านโครงสร้างออกทั้งหมด

ไฟล์ / โฟลเดอร์	ความหมาย	ตัวอย่างในโปรเจกต์
`page.tsx`	หน้าเว็บ 1 หน้า URL ตามชื่อโฟลเดอร์	`app/dashboard/page.tsx` → `/dashboard`
`layout.tsx`	กรอบที่ครอบทุกหน้าในโฟลเดอร์นั้น	`app/staff/layout.tsx` เช็กว่าเป็น staff ก่อนแสดงหน้า
`route.ts`	API มีฟังก์ชันชื่อ GET / POST / PUT / PATCH / DELETE	`app/api/auth/login/route.ts` → `POST /api/auth/login`
`[id]`	URL ที่เปลี่ยนตามค่า (dynamic)	`app/assessment/[id]/page.tsx` → `/assessment/5`
`components/`, `_lib/`	ไม่ใช่ URL เป็นที่เก็บโค้ดใช้ร่วม	`app/components/Sidebar.tsx`

หน้าแบบประเมินมี 3 ไฟล์คู่กันเสมอ

1 · หน้าทำ

assessment_stress/page.tsx

แสดงคำถามและรับคำตอบ

2 · API

api/assessments/stress/route.ts

GET คืนคำถาม · POST บันทึกผล

3 · หน้าผล

recommendation_stress/page.tsx

แสดงคะแนน ระดับ และคำแนะนำ

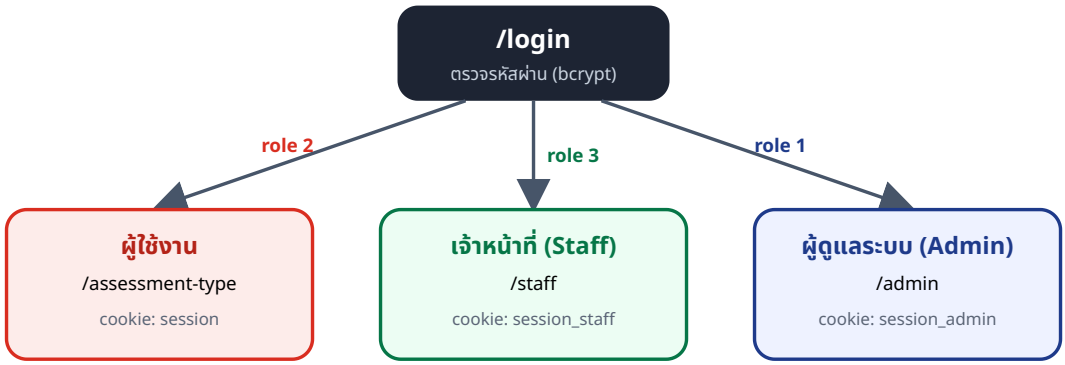
หน้าทำแบบประเมินยังมี `layout.tsx` ของตัวเองด้วย ไว้แจ้งผู้ใช้เมื่อเจ้าหน้าที่ปิดแบบประเมินนั้นอยู่

เคล็ดลับ: ถ้าเห็น URL ในเบราว์เซอร์ เช่น `/recommendation_diet` ให้เปิดไฟล์ `app/recommendation_diet/page.tsx` ได้ทันที

4 ผู้ใช้ 3 บทบาท

เข้าทางเดียวกัน แต่ไปคนละที่

ทุกคนล็อกอินที่หน้า `/login` เดียวกัน ระบบดู `role_id` ในตาราง `users` แล้วพาไปหน้าของบทบาทนั้น แต่ละบทบาทได้ cookie แยกกัน จึงเปิดคนละบทบาทคนละแท็บพร้อมกันได้



role 2

ผู้ใช้งาน

- ทำแบบประเมิน 12 แบบ
- ดูผล คำแนะนำ และ Dashboard
- ประวัติและแจ้งเตือนประเมินซ้ำ
- กรอกข้อมูลสุขภาพ และตั้งค่าบัญชี

role 3

เจ้าหน้าที่

- ภาพรวมระบบ `/staff`
- จัดการผู้ใช้ (ระงับได้ แต่ลบไม่ได้)
- แก้แบบประเมิน: ร่าง → เผยแพร่
- รายงานสถิติ ส่งออก CSV/PDF

role 1

ผู้ดูแลระบบ

- ตรวจสุขภาพระบบ `/admin`
- ดูแลฐานข้อมูล แก้ข้อมูลกำพร้า
- ดูและปิดงานข้อผิดพลาด
- สำรองข้อมูลเป็นไฟล์ JSON

ตรวจสิทธิ์ 2 ชั้น: ชั้นแรกคือ `layout.tsx` ฝั่งเบราว์เซอร์ ถ้าไม่มีสิทธิ์จะเด้งไป login ชั้นที่สองคือทุก API เรียก `requireUser` / `requireStaff` / `requireAdmin` เพื่อตรวจ cookie และ role จากฐานข้อมูลจริงอีกครั้ง ชั้นที่สองนี้คือด่านจริง

-

5 ฝั่งผู้ใช้งาน: หน้าต่าง ๆ

เรียงตามลำดับที่ผู้ใช้เจอ

1. **เข้าระบบ:** `/` → `/welcome` → `/register` → `/login` (ลืมรหัสผ่าน: `/forgot-password` → `/verify-otp` → `/reset-password`)
2. **ครั้งแรก:** `/profile` กรอกอายุ เพศ ส่วนสูง น้ำหนัก ฯลฯ เพื่อให้ระบบเติมข้อมูลในบางแบบประเมินได้เอง
3. **เลือกแบบประเมิน:** `/assessment-type` → เมนูหมวด `/assessment-menu` (NCDs), `-behavior` (พฤติกรรม), `-mental-health` (สุขภาพจิต)
4. **ทำแบบประเมิน:** หน้า `assessment_XXX` หรือ `/sleep-assessment` หรือ `/assessment/[id]` สำหรับแบบที่ staff สร้างเอง
5. **ดูผล:** หน้า `recommendation_XXX` แสดงวงกลมคะแนน ระดับ คำแนะนำ และแนะนำแบบที่ควรทำต่อ
6. **ติดตาม:** `/dashboard` (กราฟ), `/history` (ประวัติ), `/notifications` (วันประเมินซ้ำ), `/recommendation-health` (แผนดูแลรวม), `/settings` (ตั้งค่า)

แบบประเมินทั้ง 12 แบบ

หมวด	แบบประเมิน
NCDs	BMI · ความดันโลหิต · Thai CVD (หัวใจ) · เบาหวาน
พฤติกรรม	บุหรี่ · แอลกอฮอล์ · กิจกรรมทางกาย · การนอน · อาหาร
สุขภาพจิต	ความเครียด ST-5 · ซึมเศร้า 2Q → 9Q (9Q เปิดจากหน้าผล 2Q เมื่อผลออกมาว่าเสี่ยง)

ชิ้นส่วนที่ทุกหน้าใช้ร่วม (app/components/)

`Sidebar.tsx`	เมนูซ้าย 7 เมนู เช็ก session ทุกหน้า และมีปุ่มออกจากระบบ
`NotificationBell.tsx`	กระดิ่งแจ้งเตือน โหลดใหม่ทุก 60 วินาที
`recommendation/RecommendationLayout.tsx`	โครงหน้าผลที่ทั้ง 12 หน้าใช้ร่วม แก้ที่เดียวเปลี่ยนทุกหน้า
`AssessmentClosedNotice` · `HealthConsentNotice`	แจ้งเมื่อแบบประเมินถูกปิด หรือเมื่อผู้ใช้ถอนความยินยอม

6 หลังบ้าน (API)

ทุกอย่างอยู่ใต้ app/api/

โฟลเดอร์	มี API อะไร
`api/auth/`	register · login · logout · session · forgot-password · verify-otp · reset-password
`api/assessments/`	API ของแต่ละแบบประเมิน (bmi, blood-pressure, thai-cvd, diabetes, depression, smoking, alcohol, physical_activity, sleep, diet, stress, custom) + categories · sources · status
`api/assessments/_lib/`	ตัวช่วยที่ทุกแบบใช้: ตรวจข้อมูล · เช็กปิดแบบประเมิน · เช็กความยินยอม · ผูกเวอร์ชัน
`api/dashboard`, `history`, `notifications`	ข้อมูลหน้า Dashboard · ประวัติ · การแจ้งเตือน
`api/profile`, `settings/`	ข้อมูลสุขภาพ · ตั้งค่า · เปลี่ยนชื่อ/อีเมล · ส่งออก · ลบบัญชี
`api/staff/`	overview · users · assessments · categories · reports
`api/admin/`	me · system · database · errors · backup

API แบบประเมินทุกตัวทำงานเหมือนกัน

1. **GET ไม่ส่ง assessmentId** → คืนคำถาม · **GET ส่ง assessmentId** → คืนผลของผู้ใช้คนนั้นเท่านั้น
2. **POST** ตรวจตามลำดับ: อ่าน JSON → `requireUser` (login แล้วหรือยัง) → แบบประเมินถูกปิดไหม → ยินยอมข้อมูลสุขภาพไหม
3. คิดคะแนน **ฝั่ง server** จากตัวเลือกในฐานข้อมูล (ไม่เชื่อคะแนนที่หน้าเว็บส่งมา)
4. หาคำแนะนำตามระดับ แล้วบันทึก `assessment` + `assessment_answers` ใน **transaction** (สำเร็จทั้งหมด หรือยกเลิกทั้งหมด)
5. ตอบกลับ `{ success, assessment_id, total_score, risk_level }`

รหัสสถานะที่เจอบ่อย: 200 สำเร็จ · 400 ข้อมูลผิด · 401 ยังไม่ login · 403 ไม่มีสิทธิ์/ถูกปิด · 404 ไม่พบ · 500 ระบบผิดพลาด (บันทึกลงหน้า admin/errors)

7 ไฟล์ตัวช่วยใน lib/

โค้ดที่หลายไฟล์เรียกใช้ซ้ำ

ไฟล์	มีไว้ทำอะไร
`db.ts`	เปิดการเชื่อมต่อ PostgreSQL (pg Pool) ทุก API import ไฟล์นี้
`session.ts`	สร้าง/ตรวจ cookie ที่ลงลายเซ็น HMAC และ `requireUser`
`mailer.ts`	ส่งอีเมล OTP ผ่าน Gmail
`rateLimit.ts`	จำกัดการขอ OTP (3 ครั้ง/15 นาที) และการกรอกผิด (5 ครั้ง/10 นาที)
`notificationRules.ts`	กฎว่าผลระดับไหนควรประเมินซ้ำอีกกี่วัน
`riskLevel.ts`	จัดผลเป็น ปกติ / ควรระวัง / สูง และเลือกสีตามเอกสารกรมควบคุมโรค
`assessmentNames.ts`	แปลชื่อแบบประเมินจากอังกฤษเป็นไทย
`assessmentCategories*.ts`, `customAssessments.ts`	หมวดหมู่ และแบบประเมินที่ staff สร้างเอง
`consents.ts`	ข้อความขอความยินยอม PDPA
`adminAuth.ts`, `adminSchema.ts`, `errorLogger.ts`	ตรวจสิทธิ์ admin · สร้างตาราง log/backup · บันทึก error
`staff/`	`auth.ts` (requireStaff, logAccess) · `client.ts` · `assessmentVersions.ts` (ระบบเวอร์ชัน) · `riskLevels.ts` · `overviewRange.ts` · `assessmentHistory.ts`

8 ฐานข้อมูล

PostgreSQL บน Supabase — ตารางหลักที่ต้องรู้

```
roles 1──N users 1──N assessment N──1 assessment_types 1──N questions 1──N question_choices
                      |                 |
                      |                 ├── 1──N assessment_answers  (คำตอบรายข้อ)
                      |                 └── N──1 recommendation      (คำแนะนำตามระดับ)
                      ├── health_profile · user_settings · user_notifications
                      └── password_reset_tokens
```

ตาราง	เก็บอะไร
`users` / `roles`	บัญชีทุกคน + บทบาท (1 admin, 2 user, 3 staff)
`health_profile`	ข้อมูลสุขภาพพื้นฐานของผู้ใช้
`assessment_types`	รายชื่อแบบประเมิน + เปิด/ปิด + หมวด
`questions` / `question_choices`	คำถามและตัวเลือกพร้อมคะแนน
`assessment`	ผลประเมินแต่ละครั้ง (คะแนน ระดับ เวลา)
`assessment_answers`	คำตอบรายข้อของแต่ละครั้ง
`recommendation` (+ `_history`)	คำแนะนำตามระดับ และฉบับเก่าที่ระบบเก็บไว้เมื่อมีการแก้
`user_notifications`	วันครบกำหนดประเมินซ้ำ
`user_settings`	การตั้งค่า + ความยินยอม PDPA
`assessment_versions`	ฉบับร่าง/เวอร์ชันแบบประเมินของ staff
`access_logs` · `system_error_logs` · `backup_history`	log ของ staff · error ของระบบ · ประวัติสำรองข้อมูล

ทำไมแก้คำแนะนำแล้วผลเก่าไม่เปลี่ยน: trigger เก็บฉบับเก่าไว้ใน `recommendation_history` ทุกครั้งที่มีการแก้ แล้วฟังก์ชัน `recommendation_at()` ดึงคำแนะนำฉบับที่ใช้อยู่ ณ วันที่ผู้ใช้ทำแบบประเมินมาแสดง

9 ตามรอย 1 รอบการทำงาน

ตัวอย่าง: ผู้ใช้ทำแบบประเมินความเครียด



ทุกแบบประเมิน (12 แบบ) เดินตามเส้นทางเดียวกันนี้

1. `app/assessment_stress/page.tsx` เรียก `GET /api/assessments/stress` ได้คำถาม 5 ข้อ
2. ผู้ใช้ตอบครบแล้วกดส่ง หน้าเว็บส่ง `POST /api/assessments/stress` พร้อมคำตอบ
3. `app/api/assessments/stress/route.ts` ตรวจ session → แบบประเมินเปิดอยู่ไหม → ยินยอมข้อมูลสุขภาพไหม
4. คิดคะแนนจาก `question_choices` แล้วตัดระดับ (0–4 น้อย, 5–7 ปานกลาง, 8–9 มาก, 10+ มากที่สุด)
5. บันทึก `assessment` + `assessment_answers` ใน transaction แล้วตอบ `assessment_id` กลับ
6. หน้าเว็บพาไป `/recommendation_stress?assessmentId=…` แล้วดึงผลมาแสดงด้วย `RecommendationLayout`
7. ผลนี้ไปขึ้นต่อในหน้า Dashboard ประวัติ และ `notificationRules.ts` คำนวณวันที่ควรประเมินซ้ำให้

10 ความปลอดภัย

ระบบป้องกันอะไรไว้บ้าง

รหัสผ่าน

เก็บเป็น hash ด้วย bcrypt ไม่มีใครเห็นรหัสจริง แม้แต่แอดมิน

Session

cookie เป็น HttpOnly และลงลายเซ็น HMAC ถ้าแก้ค่าใน cookie ลายเซ็นจะไม่ตรง ระบบไม่ยอมรับ

ดูข้อมูลคนอื่นไม่ได้

API อ่าน userId จาก cookie และทุก SQL กรอง `user_id` ของคนที่ล็อกอินอยู่

SQL injection

ค่าที่มาจากผู้ใช้ส่งเป็น parameter `$1, $2` เสมอ ส่วนชื่อคอลัมน์ต้องผ่าน whitelist

โกงคะแนนไม่ได้

server คิดคะแนนจากฐานข้อมูลเอง ไม่ใช้คะแนนที่หน้าเว็บส่งมา

PDPA

ขอความยินยอมก่อนเก็บข้อมูล staff ไม่เห็นผลรายบุคคล กลุ่มที่น้อยกว่า 5 คนจะไม่แสดงตัวเลข และทุกการกระทำของ staff ถูกบันทึกลง log

จุดที่ยังพัฒนาต่อได้ (ตอบตรง ๆ ถ้าอาจารย์ถาม): หน้า login ยังไม่จำกัดจำนวนครั้งที่ลองรหัสผิด และ rate limit เก็บไว้ในหน่วยความจำ restart server แล้วนับใหม่

-

11 อยากแก้ X ไปที่ไฟล์ไหน

ตารางลัดสำหรับหาไฟล์

อยากแก้ / อยากดู...	ไปที่
หน้าตาหน้า login	`app/login/page.tsx`
ขั้นตอนตรวจรหัสผ่านและแยก role	`app/api/auth/login/route.ts`
เมนูซ้ายของผู้ใช้	`app/components/Sidebar.tsx`
สี / ฟอนต์ทั้งเว็บ	`app/globals.css`
คำถามของแบบประเมิน X	หน้า staff → จัดการแบบประเมิน (เก็บในตาราง `questions`) ยกเว้นแอลกอฮอล์ที่เขียนไว้ในโค้ด
สูตรคิดคะแนน / เกณฑ์ของแบบ X	`app/api/assessments/X/route.ts`
หน้าผลของแบบ X	`app/recommendation_X/page.tsx`
โครงหน้าผลที่ทุกแบบใช้	`app/components/recommendation/RecommendationLayout.tsx`
กี่วันควรประเมินซ้ำ	`lib/notificationRules.ts`
กราฟใน Dashboard	`app/dashboard/page.tsx` + `app/api/dashboard/route.ts`
อีเมล OTP	`lib/mailer.ts` + `app/api/auth/forgot-password/route.ts`
การเชื่อมต่อฐานข้อมูล	`lib/db.ts` + `.env.local` (DATABASE_URL)
หน้าและ API ของเจ้าหน้าที่	`app/staff/…` + `app/api/staff/…` + `lib/staff/…`
หน้าและ API ของแอดมิน	`app/admin/…` + `app/api/admin/…` + `lib/admin*.ts`
ตารางในฐานข้อมูล	Supabase → Table Editor หรือไฟล์ใน `sql/`

12 สรุปจำง่ายก่อนสอบ

ถ้าจำได้แค่หน้าเดียว ให้จำหน้านี้

ใช้อะไรเขียน

Next.js + React + TypeScript

ตกแต่งด้วย Tailwind CSS · ฐานข้อมูล PostgreSQL บน Supabase · ต่อด้วยไลบรารี pg

หน้าบ้าน ↔ หลังบ้าน

fetch → route.ts → JSON

อยู่ในโปรเจกต์เดียวกัน หน้าเว็บอยู่ใน `app/` ส่วน API อยู่ใน `app/api/`

หลังบ้าน ↔ ฐานข้อมูล

lib/db.ts (pg Pool)

SQL แบบ parameter และใช้ transaction ตอนบันทึกผล

3 บทบาท

user · staff · admin

login ที่เดียว แยก cookie ตาม role · ตรวจสิทธิ์ 2 ชั้น (layout + API)

ไฟล์สำคัญ

page.tsx · layout.tsx · route.ts

หน้าเว็บ · กรอบครอบหน้า · API

ตัวเลข

176 ไฟล์ · 12 แบบประเมิน

ประมาณ 51 หน้าเว็บ · 42 API · 22 ตาราง + 2 view

ประโยคเดียวอธิบายทั้งระบบ: ผู้ใช้ทำแบบประเมินบนหน้าเว็บ React หน้าเว็บส่งคำตอบไป API ของ Next.js API ตรวจสิทธิ์แล้วคิดคะแนนฝั่ง server บันทึกลง PostgreSQL บน Supabase แล้วส่งผลกับคำแนะนำกลับมาแสดง เจ้าหน้าที่ใช้หน้า /staff จัดการแบบประเมินและดูสถิติแบบไม่ระบุตัวตน ส่วนแอดมินใช้ /admin ดูแลระบบและสำรองข้อมูล